

OpenID 认证应用接入开发流程

最后修订时间：2017 年 05 月19 日 11:13 PM

目前认证系统已经支持 OpenID Connect 协议，旧版 OpenID 2.0 协议将不再进行持续更新，请新接入的站点使用新的协议接入，[戳我阅读文档](#)。

目前认证系统已经支持 OpenID Connect 协议，旧版 OpenID 2.0 协议将不再进行持续更新，请新接入的站点使用新的协议接入，[戳我阅读文档](#)。

目前认证系统已经支持 OpenID Connect 协议，旧版 OpenID 2.0 协议将不再进行持续更新，请新接入的站点使用新的协议接入，[戳我阅读文档](#)。

角色

1. OpenID Server / Identity Provider

OpenID 服务器，地址如下。注意请不要遗漏最后的英文 `/`

```
https://login.netease.com/openid/
```

2. Consumer

准备接入 OpenID 认证的 Web 应用站点。

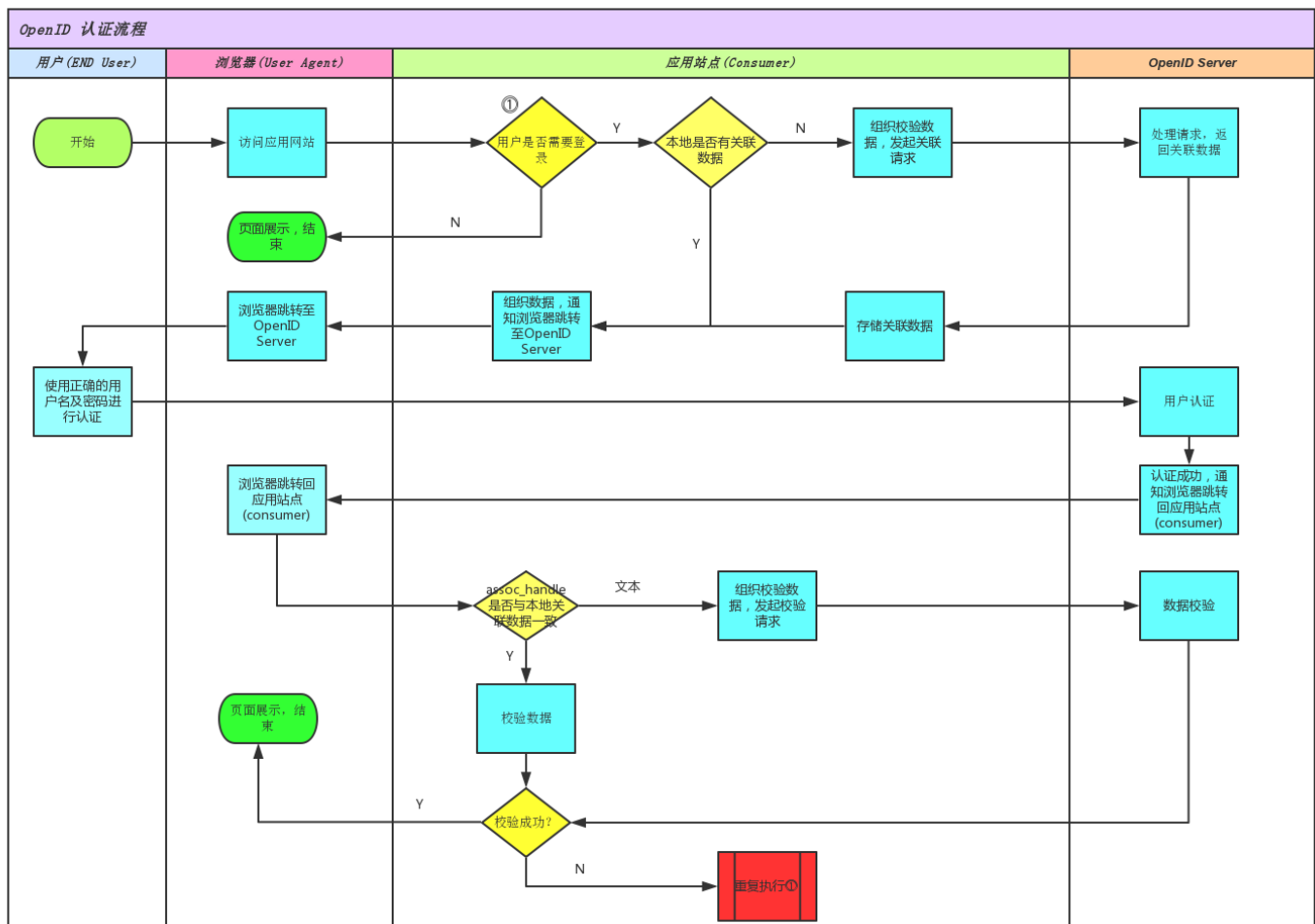
3. End User

普通用户

4. User Agent

这里通常为用户浏览器。

认证流程



1. Consumer 向 OpenID Server 发起关联请求

关联请求由应用网站 (consumer) 向 [OpenID Server](#) 发起一个直接的 Http POST 请求。请求参数如下 (表 1)：

参数名称	参数值
openid.mode	associate
openid.assoc_type	建议使用 <code>HMAC-SHA256</code>
openid.session_type	由于我们已经使用https，这里直接使用固定字符串 <code>no-encryption</code> 即可

使用 HTTP POST 向 [OpenID Server](#) 提交 `关联数据`，[OpenID Server](#) 将返回如下内容：

```

assoc_handle:{HMAC-SHA256}{52ae73e1}{JhSjeA==}
assoc_type:HMAC-SHA256
expires_in:86400
mac_key:1V0mT0G6sZMmCrftbQbnTP3+fKgKo/tB0Gc+RG5vNQu=
  
```

- 1. HTTP状态码一定为200。
- 2. 返回的内容以换行符分割，内容为键值对形式。
- 3. 这些信息会贯穿于后续的认证过程，并且属于敏感信息，请不要记录在 `cookie` 信息中。
- 4. 这些数据请存储至本地服务器的合适位置（如文件或数据库）。
- 5. `expires_in` 单位为 `秒`。存储时，请记录当前时间，以上关联信息将在 86400 秒后过期，过期后，需要重新进行关联。
- 6. 关联信息未过期时，该关联信息可重复使用，无须每次认证都发起关联请求。

2. Consumer 通知 User Agent 跳转

此步骤 Consumer 将必须的参数组织编码为 `URL query string` 并向 User Agent 发送跳转至 该 URL 的动作，必须的参数如下（表2）：

参数名称	参数值
<code>openid.ns</code>	http://specs.openid.net/auth/2.0
<code>openid.mode</code>	<code>checkid_setup</code>
<code>openid.assoc_handle</code>	第一步 <code>1. 关联请求</code> 中获取的 <code>assoc_handle</code> 值。
<code>openid.return_to</code>	用户在 OpenID Server 认证成功后，跳转回应用站点的目标URL地址。
<code>openid.claimed_id</code>	http://specs.openid.net/auth/2.0/identifier_select
<code>openid.identity</code>	http://specs.openid.net/auth/2.0/identifier_select
<code>openid.realm</code>	应用站点的URL地址，通常这个URL要能覆盖 <code>openid.return_to</code> 设定的URL。如 <code>openid.realm</code> 为： <code>http://demo.163.com/login/</code> ，那么 <code>openid.return_to</code> 一定需要以 <code>http://demo.163.com/login/</code> 开始。
<code>openid.ns.sreg</code>	http://openid.net/extensions/sreg/1.1
<code>openid.sreg.required</code>	可选项为nickname、email、fullname，分别表示昵称，邮件地址，全名（用户中文名，i 可能为空）。如果你要请求这三个中的多个，需要以英文“,”分隔，如设置为“nickname,email”，表示希望取得用户的昵称和邮件地址

假设我需从站点（`openid.realm`）

```
http://220.181.28.15:8831
```

发起认证请求，当用户在OpenID认证成功后，可以返回（`openid.return_to`）

```
http://220.181.28.15:8831/consumer/
```

并且我希望可以拿到用户基本信息里（openid.sreg.required）的如下信息：

```
nickname
email
fullname
```

假设，我在第一步 Consumer 向OpenID Server发起关联请求 那里，获取到的关联数据如下：

key	value
assoc_handle	{HMAC-SHA256}{52ae73e1}{JhSjeA==}
mac_key	1V0mT0G6sZMmCrftbQbnTP3+fKgKo/tB0Gc+RG5vNqU=

那么，综合 表2 必须的参数，我的应用网站需要通知用户跳转至如下URL：

```
https://login.netease.com/openid/?openid.mode=checkid_setup&openid.ns=http%3A%2F%2Fspecs.openid.net%2Fauth%2F2.0&openid.realm=http%3A%2F%2F220.181.28.15%3A8831%2F&openid.sreg.required=nickname%2Cemail%2Cfullname&openid.assoc_handle=%7BHMAC-SHA256%7D%7B52ae73e1%7D%7BJhSjeA%3D%3D%7D&openid.return_to=http%3A%2F%2F220.181.28.15%3A8831%2Fconsumer%2F&openid.ns.sreg=http%3A%2F%2Fopenid.net%2Fextensions%2Fsreg%2F1.1&openid.identity=http%3A%2F%2Fspecs.openid.net%2Fauth%2F2.0%2Fidentifier_select&openid.claimed_id=http%3A%2F%2Fspecs.openid.net%2Fauth%2F2.0%2Fidentifier_select
```

上面的链接为编码成 [URL query string](#) 的链接，为了方便查看，上面的URL，在不做url编码的情况下应该是以下内容：

```
https://login.netease.com/openid/?openid.mode=checkid_setup&openid.ns=http://specs.openid.net/auth/2.0&openid.realm=http://220.181.28.15:8831/&openid.sreg.required=nickname,email,fullname&openid.assoc_handle={HMAC-SHA256}{52ae73e1}{JhSjeA==}&openid.return_to=http://220.181.28.15:8831/consumer/&openid.ns.sreg=http://openid.net/extensions/sreg/1.1&openid.identity=http://specs.openid.net/auth/2.0/identifier_select&openid.claimed_id=http://specs.openid.net/auth/2.0/identifier_select
```

应用通知URL跳转到如上URL后，后面的过程就是用户、User Agent与 [OpenID Server](#) 的交互过程了，直到用户在 [OpenID Server](#) 认证成功，并跳转回应用设置的（ openid.return_to ）后，进入下一个阶段：数据校验。

3. 数据校验

当用户访问上面的URL，并成功验证后，根据应用设置的 openid.return_to ， [OpenID Server](#) 将通知

用户跳转回如下地址：

```
http://220.181.28.15:8831/consumer/?openid.assoc_handle={HMAC-SHA256}{52ae73e1}{JhSjeA%3D%3D}&openid.ax.mode=fetch_response&openid.claimed_id=https%3A%2F%2Flogin.netease.com%2Fopenid%2Fchenxs%2F&openid.identity=https%3A%2F%2Flogin.netease.com%2Fopenid%2Fchenxs%2F&openid.mode=id_res&openid.ns=http%3A%2F%2Fspecs.openid.net%2Fauth%2F2.0&openid.ns.ax=http%3A%2F%2Fopenid.net%2Fsrv%2Fax%2F1.0&openid.ns.sreg=http%3A%2F%2Fopenid.net%2Fextensions%2Fsreg%2F1.1&openid.op_endpoint=https%3A%2F%2Flogin.netease.com%2Fopenid%2F&openid.response_nonce=2013-12-16T03%3A34%3A49ZrhjPaT&openid.return_to=http%3A%2F%2F220.181.28.15%3A8831%2Fconsumer%2F&openid.sig=iKfshlYzRgH5Mo80TrEV2RQvNk88y%2BTQCYBjHlv7kGk%3D&openid.signed=assoc_handle%2Cax.mode%2Cclaimed_id%2Cidentity%2Cmode%2Cns%2Cns.ax%2Cns.sreg%2Cop_endpoint%2Cresponse_nonce%2Creturn_to%2Csigned%2Csreg.email%2Csreg.fullname%2Csreg.nickname&openid.sreg.email=chenxs%40corp.netease.com&openid.sreg.fullname=陈小生&openid.sreg.nickname=chenxs
```

为了方便查看，对上面URL里的参数进行分解：

```
openid.op_endpoint=https://login.netease.com/openid/  
openid.sig=iKfshlYzRgH5Mo80TrEV2RQvNk88y+TQCYBjHlv7kGk=  
openid.return_to=http://220.181.28.15:8831/consumer/  
openid.ns=http://specs.openid.net/auth/2.0  
openid.sreg.fullname=陈小生  
openid.response_nonce=2013-12-16T03:34:49ZrhjPaT  
openid.ax.mode=fetch_response  
openid.claimed_id=https://login.netease.com/openid/chenxs/  
openid.ns.sreg=http://openid.net/extensions/sreg/1.1  
openid.sreg.nickname=chenxs  
openid.ns.ax=http://openid.net/srv/ax/1.0  
openid.signed=assoc_handle,ax.mode,claimed_id,identity,mode,ns,ns.ax,ns.sreg,op_endpoint,response_nonce,return_to,signed,sreg.email,sreg.fullname,sreg.nickname  
openid.assoc_handle={HMAC-SHA256}{52ae73e1}{JhSjeA==}  
openid.mode=id_res  
openid.identity=https://login.netease.com/openid/chenxs/  
openid.sreg.email=chenxs@corp.netease.com
```

我们可以看到其中一个参数为

```
openid.signed=assoc_handle,ax.mode,claimed_id,identity,mode,ns,ns.ax,ns.sreg,op_endpoint,response_nonce,return_to,signed,sreg.email,sreg.fullname,sreg.nickname
```

这个参数表示，OpenID Server对这些key及其对应的值，做了签名。签名的键值如下：

```
openid.sig=iKfshlYzRgH5Mo80TrEV2RQvNk88y+TQCYBjHlv7kGk=
```

即，OpenID Server对这些内容做了签名后，算出来的值为

```
iKfshlYzRgH5Mo80TrEV2RQvNk88y+TQCYBjHlv7kGk=
```

那怎么校验我们得到的数据签名值是否与OpenID Server返回的一致呢,首先, 我们需要用到第一步关联数据时, 取得的mac_key

```
1V0mT0G6sZMmCrftbQbnTP3+fKgKo/tB0Gc+RG5vNqU=
```

其次, 我们将 `openid.signed` 得到的值以英文','分隔, 并从返回的URL里, 获取到对应key的值, 如 `openid.signed` 里, 第一个值为 `assoc_handle`, 那么, 在上述URL里, 他的Key是 `openid.assoc_handle`, 即:

```
openid.assoc_handle={HMAC-SHA256}{52ae73e1}{JhSjeA==}
```

逐一将这些KEY对应的值拿出来, 并按如下格式进行组织:

```
key1:value1  
key2:value2
```

即, 键值之间通过英文':'连接, 多个键值对间使用换行符连接。"注意, 换行是"`\n`", 非"`\r\n`", 且中文一定要使用UTF-8编码

通常, 在取这些值的时候, 应用就可以做两个最初步的校验:

I.

`openid.mode` 是否为 `id_res`

`openid.identity` 是否以 `https://login.netease.com/openid/` 开始

`openid.claimed_id` 是否以 `https://login.netease.com/openid/` 开始

通常, 这两个中的任何一个不满足, 你都可以直接认为他认证不成功。

II.

`openid.assoc_handle` 是否与 `1. Consumer 向 OpenID Server 发起关联请求` 获取的 `assoc_handle` 一致, 如果不一致, 请直接看 `4. 依赖OpenID Server的数据校验`

这两个最基本的校验满足后, 将需要验证签名的key及其对应的值进行组织, 得到如下内容: :

```
assoc_handle:{HMAC-SHA256}{52ae73e1}{JhSjeA==}  
ax.mode:fetch_response  
claimed_id:https://login.netease.com/openid/chenxs/  
identity:https://login.netease.com/openid/chenxs/  
mode:id_res  
ns:http://specs.openid.net/auth/2.0  
ns.ax:http://openid.net/srv/ax/1.0  
ns.sreg:http://openid.net/extensions/sreg/1.1  
op_endpoint:https://login.netease.com/openid/  
response_nonce:2013-12-16T03:34:49ZrhjPaT  
return_to:http://220.181.28.15:8831/consumer/  
signed:assoc_handle,ax.mode,claimed_id,identity,mode,ns,ns.ax,ns.sreg,op_endpoint,  
response_nonce,return_to,signed,sreg.email,sreg.fullname,sreg.nickname  
sreg.email:chenxs@corp.netease.com  
sreg.fullname:陈小生  
sreg.nickname:chenxs
```

注意最后的 `sreg.nickname:chenxs` 后面也是有一个"\n"的，然后使用第一步获取到的 `mac_key` 对如上内容进行 `hmac-sha256` 计算 digest，并使用base64进行encode，即得到上述内容的签名值。

这里以Python代码做为示例：

```

>>> import hashlib
>>> import hmac
>>> import base64
>>> mac_key = "1V0mT0G6sZMmCrftbQbnTP3+fKgKo/tB0Gc+RG5vNqU="
>>>
>>> b64decode_mac_key = base64.b64decode(mac_key)
>>>
>>> openid_response_content="""assoc_handle:{HMAC-SHA256}{52ae73e1}{JhSjeA==}
... ax.mode:fetch_response
... claimed_id:https://login.netease.com/openid/chenxs/
... identity:https://login.netease.com/openid/chenxs/
... mode:id_res
... ns:http://specs.openid.net/auth/2.0
... ns.ax:http://openid.net/srv/ax/1.0
... ns.sreg:http://openid.net/extensions/sreg/1.1
... op_endpoint:https://login.netease.com/openid/
... response_nonce:2013-12-16T03:34:49ZrhjPaT
... return_to:http://220.181.28.15:8831/consumer/
... signed:assoc_handle,ax.mode,claimed_id,identity,mode,ns,ns.ax,ns.sreg,op_endpoint,resp
... sreg.email:chenxs@corp.netease.com
... sreg.fullname:陈小生
... sreg.nickname:chenxs
... """
>>> hmac_sha256_digest = hmac.new(b64decode_mac_key, openid_response_content, hashlib.sha256)
>>> base64_digest = base64.b64encode(hmac_sha256_digest)
>>> print base64_digest
iKfshlYzRgH5Mo80TrEV2RQvNk88y+TQCYBjH1v7kGk=
>>>

```

计算后得到的base64_digest若与 `openid.sig` 一致，校验签名成功，否则你可以认为认证不成功。校验成功后，你就可以进行你应用站点的登录逻辑了（比如判断是否有这个用户，是否允许他登录等等，并记录session等等）

4. 向OpenID Server发起数据校验请求

这一步只在第三步数据校验时，当第一步获取的 `assoc_handle` 值与用户跳转回来后，提交的 `assoc_handle` 值不一致时才进行。

该请求只要将 [OpenID Server](#) 返回的数据，修改 `openid.mode` 为固定字符串

`check_authentication`，并将所有数据原样以Http Post方式发回给 [OpenID Server](#) 即可。

看回第三步我们取出来的OpenID Server返回的数据

```
openid.op_endpoint=https://login.netease.com/openid/  
openid.sig=iKfshlYzRgH5Mo80TrEV2RQvNk88y+TQCYBjHlv7kGk=  
openid.return_to=http://220.181.28.15:8831/consumer/  
openid.ns=http://specs.openid.net/auth/2.0  
openid.sreg.fullname=陈小生  
openid.response_nonce=2013-12-16T03:34:49ZrhjPaT  
openid.ax.mode=fetch_response  
openid.claimed_id=https://login.netease.com/openid/chenxs/  
openid.ns.sreg=http://openid.net/extensions/sreg/1.1  
openid.sreg.nickname=chenxs  
openid.ns.ax=http://openid.net/srv/ax/1.0  
openid.signed=assoc_handle,ax.mode,claimed_id,identity,mode,ns,ns.ax,ns.sreg,op_endpoint,response_nonce,return_to,signed,sreg.email,sreg.fullname,sreg.nickname  
openid.assoc_handle={HMAC-SHA256}{52ae73e1}{JhSjeA==}  
openid.mode=id_res  
openid.identity=https://login.netease.com/openid/chenxs/  
openid.sreg.email=chenxs@corp.netease.com
```

注意：首先请检查 **openid.identity** 和 **openid.claimed_id** 值均以 <https://login.netease.com/openid/>，否则你可以认为这是一个非法的请求，直接拒绝登录。

把上面的 `openid.mode` 值替换为字符串 `check_authentication`，然后将所有数据POST回 [OpenID Server](#) 即可。如：

```
openid.op_endpoint=https%3A%2F%2Flogin.netease.com%2Fopenid%2F&openid.sig=iKfshlYzRgH5Mo80TrEV2RQvNk88y%2BTQCYBjHlv7kGk%3D&openid.return_to=http%3A%2F%2F220.181.28.15%3A8831%2Fconsumer%2F&openid.ns=http%3A%2F%2Fspecs.openid.net%2Fauth%2F2.0&openid.sreg.fullname=%E9%99%88%E5%B0%8F%E7%94%9F&openid.response_nonce=2013-12-16T03%3A34%3A49ZrhjPaT&openid.ax.mode=fetch_response&openid.claimed_id=https%3A%2F%2Flogin.netease.com%2Fopenid%2Fchenxs%2F&openid.ns.sreg=http%3A%2F%2Fopenid.net%2Fextensions%2Fsreg%2F1.1&openid.sreg.nickname=chenxs&openid.ns.ax=http%3A%2F%2Fopenid.net%2Fsrv%2Fax%2F1.0&openid.signed=assoc_handle%2Cax.mode%2Cclaimed_id%2Cidentity%2Cmode%2Cns%2Cns.ax%2Cns.sreg%2Cop_endpoint%2Cresponse_nonce%2Creturn_to%2Csigned%2Csreg.email%2Csreg.fullname%2Csreg.nickname&openid.assoc_handle=%7BHMAC-SHA256%7D%7B52ae73e1%7D%7BJhSjeA%3D%3D%7D&openid.mode=check_authentication&openid.identity=http%3A%2F%2Flogin.netease.com%2Fopenid%2Fchenxs%2F&openid.sreg.email=chenxs%40corp.netease.com
```

服务器将返回如下内容：

```
is_valid:true  
ns:http://specs.openid.net/auth/2.0
```

只要返回的内容中，其中一行为 `is_valid:true`，即可认为校验成功，否则应用可以直接认为用户认证失败。

Examples

Python

```
https://login.netease.com/download/pure_python_openid_demo.py
```

Java

```
https://login.netease.com/download/openid_for_java.zip
```

认证与安全

可区分公司内外网来源，并支持多种简单认证及将军令加强版认证，具体可参考：[站点接入](#)，将军令绑定请参考：[将军令绑定](#)

1. 虽然 OpenID 可以区分用户来源内外网，但应用也需要在自己的服务器或应用上，限制只对允许访问应用的来源IP开放，以进一步提高安全性。
2. 接入OpenID之后，所有成功通过OpenID校验的用户都可能登录你的应用，所以请在你的应用做好权限控制，只允许白名单的用户登录你的应用。

优化建议

1. 开源的 openid 插件，部份插件会发送大量无用的 openid.ax 请求给 openid server，这将带来的问题是，完成用户校验，需要返回应用站点时，可能出现 openid.return_to 长度超过2K甚至4K的情况，而部份浏览器，如 MSIE 和 safari 会有 HTTP GET 长度为 2K 的限制，导致用户不能正常访问服务或出现不友好提示。

建议在使用开源插件的基础上，要视情况做出一定的精简。

FAQ

OpenID通讯过程，中文使用什么编码？

UTF-8。如果你的服务器环境确实有非常复杂的编码问题，建议你不要获取用户fullname（真实中文姓名）、openid.ax.type.dep（部门）。

集群如何保存状态信息

对于集群，如果你希望通过第一步关联（associate）获取 `assoc_handle` 信息，并自己在服务器本地进行校验签名信息的话，建议你的关联信息要存储在所有集群内机器都可以读取到的地方，如Memcache，MySQL等。

如何申请接入

请使用 <https://login.netease.com/sitemgmt> 进行自助接入。